Introduction to Haskell Programming

More IO

S P Suresh

Chennai Mathematical Institute

Haskell @ CMI

Combining pure functions and `IO` actions

- Haskell type system allows us to combine pure functions and actions in a safe manner

Combining pure functions and `IO` actions

- Haskell type system allows us to combine pure functions and actions in a safe manner
- No mechanism to execute an action inside a pure function

Combining pure functions and `IO` actions

- Haskell type system allows us to combine pure functions and actions in a safe manner
- No mechanism to execute an action inside a pure function
- But pure functions can be used as subroutines inside actions

Combining pure functions and `IO` actions

- Haskell type system allows us to combine pure functions and actions in a safe manner
- No mechanism to execute an action inside a pure function
- But pure functions can be used as subroutines inside actions
- IO is performed by an action only if it is executed from within another action

Combining pure functions and `IO` actions

- Haskell type system allows us to combine pure functions and actions in a safe manner
- No mechanism to execute an action inside a pure function
- But pure functions can be used as subroutines inside actions
- IO is performed by an action only if it is executed from within another action
- `main` is where all the action begins

IO example, repetition

- Read a line and print it out ten times

```
main = do
    inp ← getLine
    printOften 10 inp

printOften :: Int → String → IO ()
printOften 0 str = return ()
printOften n str = do
    putStrLn str
    printOften (n-1) str
```

IO example, repetition – `getLine`

- Here is a possible implementation of `getLine`

```
getLine :: IO String
getLine = do
    c ← getChar
    if (c == '\n') then return ""
    else do
            cs ← getLine
            return (c:cs)
```

IO example, repetition – `getLine`

- Here is a possible implementation of `getLine`

```
getLine :: IO String
getLine = do
    c ← getChar
    if (c == '\n') then return ""
    else do
            cs ← getLine
            return (c:cs)
```

- Note the use of `return` and the recursion

Repetition and IO, another example

- Repeat an IO action n times

```
ntimes :: Int → IO () → IO ()
ntimes 0 a = return ()
ntimes n a = do
    a
    ntimes (n-1) a
```

Repetition and IO, another example

- Repeat an IO action n times

```
ntimes :: Int → IO () → IO ()
ntimes 0 a = return ()
ntimes n a = do
    a
    ntimes (n-1) a
```

- Read and print 10 lines

```
main = ntimes 10 (do
    inp ← getLine
    putStrLn inp
)
```

Reading types other than `String`

- The function `readLn` reads the value of any type `a` that is an instance of the typeclass `Read`

```
readLn :: Read a ⇒ IO a
```

Reading types other than `String`

- The function `readLn` reads the value of any type `a` that is an instance of the typeclass `Read`

```
readLn :: Read a ⇒ IO a
```

- All basic types (`Int`, `Bool`, `Char`, etc.) are instances of `Read`

Reading types other than `String`

- The function `readLn` reads the value of any type `a` that is an instance of the typeclass `Read`

```
readLn :: Read a ⇒ IO a
```

- All basic types (`Int`, `Bool`, `Char`, etc.) are instances of `Read`
- Basic type constructors also preserve readability

Reading types other than `String`

- The function `readLn` reads the value of any type `a` that is an instance of the typeclass `Read`

```
readLn :: Read a ⇒ IO a
```

- All basic types (`Int`, `Bool`, `Char`, etc.) are instances of `Read`
- Basic type constructors also preserve readability
- `[Int]`, `(Int, Bool, Char)`, etc. are also instances of `Read`

Reading types other than `String`

- The function `readLn` reads the value of any type `a` that is an instance of the typeclass `Read`

```
readLn :: Read a ⇒ IO a
```

- All basic types (`Int`, `Bool`, `Char`, etc.) are instances of `Read`
- Basic type constructors also preserve readability
- `[Int]`, `(Int, Bool, Char)`, etc. are also instances of `Read`
- Syntax to read an integer

```
inp ← readLn :: IO Int
```

Reading integers, example

- Read non-negative integers and print the corresponding factorial

Reading integers, example

- Read non-negative integers and print the corresponding factorial
- Stop if you encounter a negative number

```
import System.IO ( hFlush, stdout )
main = do
    putStr "Enter a number: "; hFlush stdout
    n ← readLn :: IO Int
    if n < 0
    then do
        putStrLn "Exiting. Good bye!"
        return ()
    else do
        putStrLn $ "The factorial of " ++ show n ++ " = " ++ show (fac n)
        main
```

Reading integers, example

- Read non-negative integers (one on each line) and print the list at the end

Reading integers, example

- Read non-negative integers (one on each line) and print the list at the end
- Stop if you encounter a negative number

```
main = do
    xs ← readIntList
    print xs

readIntList :: IO [Int]
readIntList = do
    x ← readLn :: IO Int
    if (x < 0) then return []
    else do
        xs ← readIntList
        return (x:xs)
```

Reading a list of integers

- Read a list of non-negative integers and print out list at the end

```
main = do {xs ← readIntList; print xs;}
readIntList :: IO [Int]
readIntList = do
    x ← readLn :: IO Int
    if (x < 0)
    then return []
    else do
        xs ← readIntList
        return (x:xs)
```

Reading a list of integers

- Read a list of non-negative integers and print out list at the end

```
main = do {xs ← readIntList; print xs;}
readIntList :: IO [Int]
readIntList = do
    x ← readLn :: IO Int
    if (x < 0)
    then return []
    else do
        xs ← readIntList
        return (x:xs)
```

- What if we want to signal the end of input by some other means?

Reading a list of integers

- Read a list of non-negative integers and print out list at the end

```
main = do {xs ← readIntList; print xs;}
readIntList :: IO [Int]
readIntList = do
    x ← readLn :: IO Int
    if (x < 0)
    then return []
    else do
        xs ← readIntList
        return (x:xs)
```

- What if we want to signal the end of input by some other means?
- Say, input is from a file and we process each line till the file ends

Reading a list of integers

- Use `isEOF` (requires `import System.IO`)

```
import System.IO
main = do
    xs ← readIntList
    print xs
readIntList = do
    exitCond ← isEOF
    if exitCond
    then return []
    else do
        x ← readLn :: IO Int
        xs ← readIntList; return (x:xs)
```

Reading a list of integers

- Use `isEOF` (requires `import System.IO`)

```
import System.IO
main = do
    xs ← readIntList
    print xs
readIntList = do
    exitCond ← isEOF
    if exitCond
    then return []
    else do
        x ← readLn :: IO Int
        xs ← readIntList; return (x:xs)
```

- `isEOF` returns `True` when end of file (or **Ctrl-D**) is reached

Repetition using `forever`

- Repeatedly read a list of integers on each line and print its reverse

Repetition using `forever`

- Repeatedly read a list of integers on each line and print its reverse
- Use `forever` to repeatedly perform an action (requires `import Control.Monad`)

Repetition using `forever`

- Repeatedly read a list of integers on each line and print its reverse
- Use `forever` to repeatedly perform an action (requires `import Control.Monad`)
- Use `exitSuccess` to exit the loop (requires `import System.Exit`)

Repetition using `forever`

- Repeatedly read a list of integers on each line and print its reverse
- Use `forever` to repeatedly perform an action (requires `import Control.Monad`)
- Use `exitSuccess` to exit the loop (requires `import System.Exit`)
- Check when to exit using `isEOF`

Repetition using `forever`

- Repeatedly read a list of integers on each line and print its reverse

```
import System.IO
import System.Exit
import Control.Monad
main = forever (do
    exitCond ← isEOF
    if exitCond then exitSuccess else do
        inList ← readLn :: IO [Int]
        print (reverse inList)
)
```

Repetition using `forever`

- Convenient to use `when` along with `forever` to handle the exit case (requires `import Control.Monad`)

```
import System.IO
import System.Exit
import Control.Monad
main = forever (do
    exitCond ← isEOF
    when exitCond exitSuccess
    inList ← readLn :: IO [Int]
    print (reverse inList)
)
```

Rudimentary file IO

- Simplest way to read from files and write into files is by **input/ output redirection**

Rudimentary file IO

- Simplest way to read from files and write into files is by **input/ output redirection**
- Usual input is from **standard input** (which is the keyboard)

Rudimentary file IO

- Simplest way to read from files and write into files is by **input/ output redirection**
- Usual input is from **standard input** (which is the keyboard)
- Read input from a file by input redirection

```
./myprogram < inputfile
```

Rudimentary file IO

- Simplest way to read from files and write into files is by **input/ output redirection**
- Usual input is from **standard input** (which is the keyboard)
- Read input from a file by input redirection

```
./myprogram < inputfile
```

- Usual output is to **standard output** (which is the screen)

Rudimentary file IO

- Simplest way to read from files and write into files is by **input/ output redirection**
- Usual input is from **standard input** (which is the keyboard)
- Read input from a file by input redirection

```
./myprogram < inputfile
```

- Usual output is to **standard output** (which is the screen)
- Send output to a file by output redirection

```
./myprogram > outputfile
```

Rudimentary file IO

- Simplest way to read from files and write into files is by **input/ output redirection**
- Usual input is from **standard input** (which is the keyboard)
- Read input from a file by input redirection

```
./myprogram < inputfile
```

- Usual output is to **standard output** (which is the screen)
- Send output to a file by output redirection

```
./myprogram > outputfile
```

- Can combine the two:

```
./myprogram < inputfile > outputfile
```

`forM` and `forM_` – repeat for each element in a list

- Given a list of strings, print the reverse of each, one on a line

```
import Control.Monad
printRevs :: [String] → IO [()]
printRevs xs = forM xs (putStrLn . reverse)

printRevs' :: [String] → IO ()
printRevs' xs = forM_ xs (putStrLn . reverse)
```

Repeat an action a fixed number of times

- Ask user to input a name and print its reverse, ten times

```
import Control.Monad
getAndPut :: IO ()
getAndPut = do { s ← getLine; putStrLn (reverse s); }

tenTimes, tenTimes' :: IO ()
tenTimes = forM_ [1..10] (\_ → getAndPut)
tenTimes' = replicateM_ 10 getAndPut

tenTimes'' :: IO [String]
tenTimes'' = replicateM 10 getLine
```

`randomIO` and `randomRIO` – generate random values

- `randomIO :: IO a` produces a value of type `a`

`randomIO` and `randomRIO` – generate random values

- `randomIO :: IO a` produces a value of type `a`
- `randomRIO (l, u) :: IO a` – a random value of type `a` from `[l .. u]`

`randomIO` and `randomRIO` – generate random values

- `randomIO :: IO a` produces a value of type `a`
- `randomRIO (l, u) :: IO a` – a random value of type `a` from `[l .. u]`
- Combine with `replicateM` to produce random lists

```
import System.Random ( randomIO, randomRIO )
import Control.Monad ( replicateM )

randIntList :: Int → IO [Int]
randIntList sz = replicateM sz randomIO

randString :: Int → IO String
randString sz = replicateM sz (randomRIO ('a', 'z'))
```